Membrane Gas Separation Simulation with OpenFOAM: A Short Practical Course

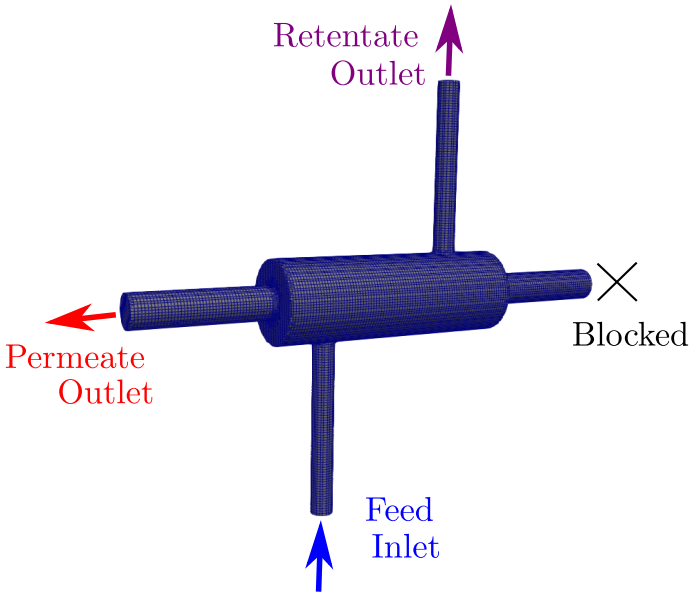
Lecture 03

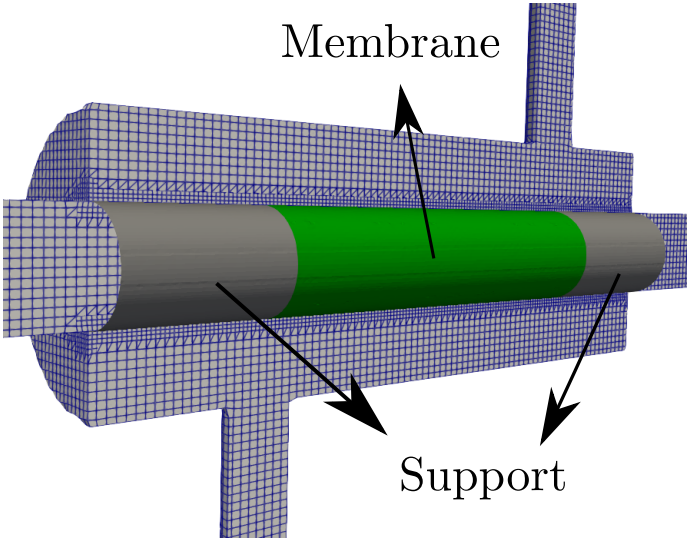
Professor Adriano Possebon Rosa (aprosa@unb.br)

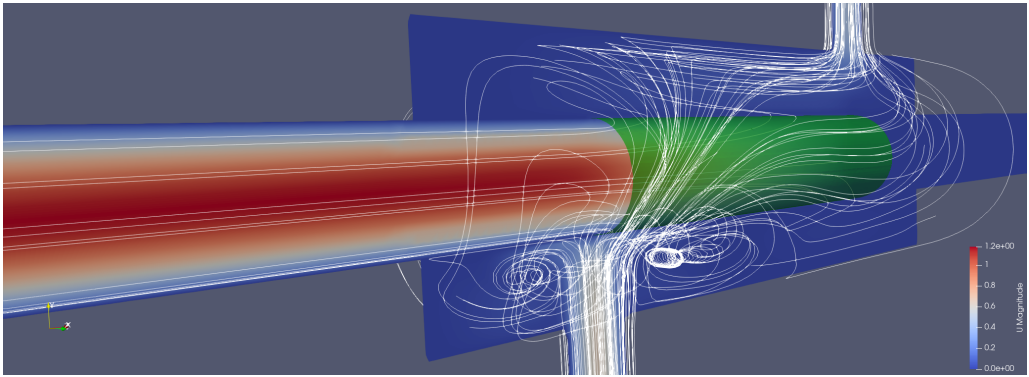
Energy and Environment Laboratory (lea.unb.br)

University of Brasilia

Goal: simulate a membrane gas separation module using Computational Fluid Dynamics (CFD).


Retentate
Outlet
Blocked
Permeate
Outlet
Feed
Inlet


Membrane
Support


1.2e+00
1
0.8
0.6
0.4
0.2
0.0e+00
U Magnitude

Lectures:

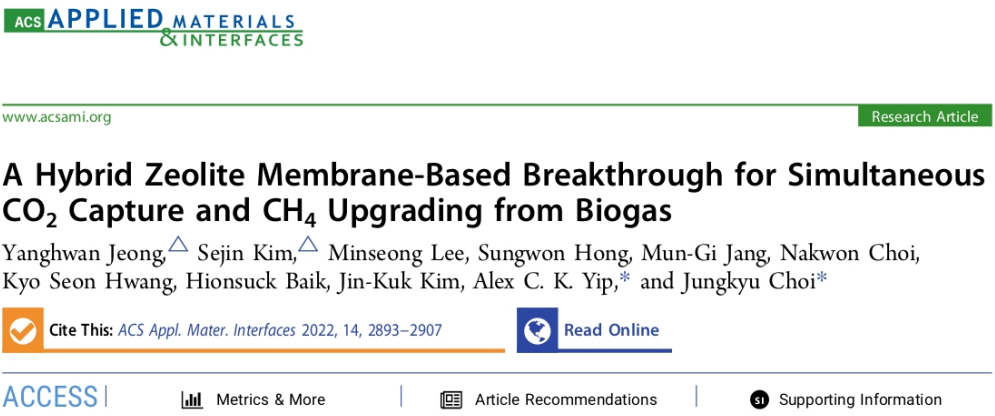
- **Lecture 00:** Membrane Gas Separation; Governing Equations; CFD Basics; Finite Volume Method; OpenFOAM; Simulation: Lid-Driven Cavity.
- **Lecture 01:** Governing Equations (Basic Problems); Simulation: Parallel Plates; Mesh Basics; Mesh Generation with Gmsh; Simulation: T-Junction; Baffles.
- **Lecture 02:** Gas Flow Simulation; Baffles (Membranes); Simulation: T-Junction with Membrane; SnappyHexMesh; Simulation: 3D T-Junction with Membrane and SHM.
- **Lecture 03:** Running the Complete Simulation (Membrane Module); Simulation Setup; Boundary Conditions; Initial Conditions; Simulation and Experiment; Parallelization; Post Process.

In this lecture, we will simulate a membrane gas-separation module using OpenFOAM.

The case is based on an experiment from Jeong et al.

Table of Contents

Link for the paper.



ACS APPLIED MATERIALS & INTERFACES

www.acsami.org

Research Article

A Hybrid Zeolite Membrane-Based Breakthrough for Simultaneous CO_2 Capture and CH_4 Upgrading from Biogas

Yanghwan Jeong,△ Sejin Kim,△ Minseong Lee, Sungwon Hong, Mun-Gi Jang, Nakwon Choi, Kyo Seon Hwang, Hionsuck Baik, Jin-Kuk Kim, Alex C. K. Yip,* and Jungkyu Choi*

Cite This: *ACS Appl. Mater. Interfaces* 2022, 14, 2893−2907

Read Online

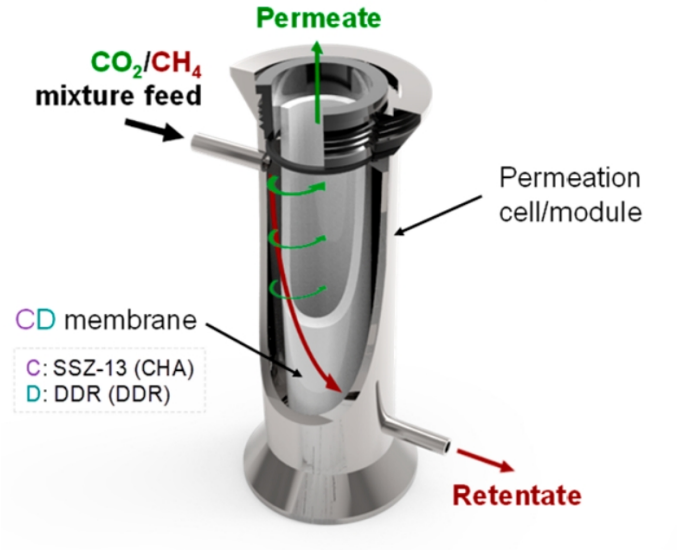
ACCESS | Metrics & More | Article Recommendations | Supporting Information

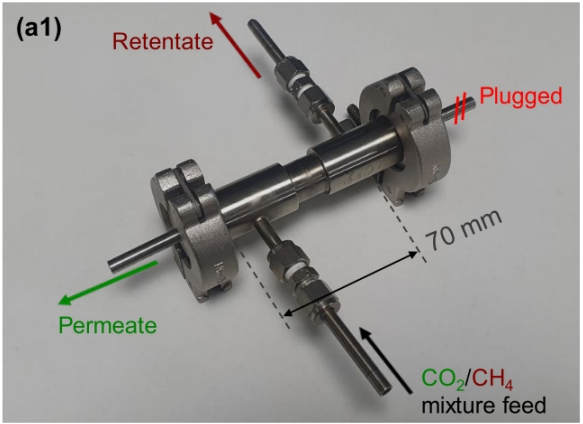
There is also a Supporting Information paper. Link.

Supporting Information

A hybrid zeolite membrane-based breakthrough for simultaneous CO_2 capture and CH_4 upgrading from biogas

Yanghwan Jeong,[1,†] Sejin Kim,[1,†] Minseong Lee,[1] Sungwon Hong,[1] Mun-Gi Jang,[2] Nakwon Choi,[3,4] Kyo Seon Hwang,[5] Hionsuck Baik,[6] Jin-Kuk Kim,[2] Alex C.K. Yip,[7,*] and Jungkyu Choi[1,*]


Permeate
CO_2/CH_4
mixture feed
Permeation
cell/module
CD membrane
C: SSZ-13 (CHA)
D: DDR (DDR)
Retentate


(a1)
Retentate
Plugged
70 mm
Permeate
CO_2/CH_4
mixture feed

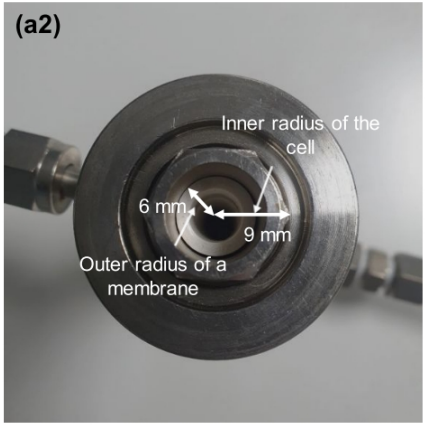

(a2)
Inner radius of the cell
6 mm
9 mm
Outer radius of a membrane

Table from the Supporting Information paper.

Table S1. Numerical values of the feed flow rates and the flow rates and compositions of CO_2 and CH_4 on the permeate and retentate sides with respect to the separation performance of CD-1-Cell under dry conditions at 30 °C shown in Figure 4a2,b2.

Feed			**Permeate side**				**Retentate side**			
Total flow rate ($mL \cdot min^{-1}$)	Molar flow rate		Molar flow rate		Composition		Molar flow rate		Composition	
	CO_2 ($mol \cdot s^{-1}$) $\times 10^4$	CH_4 ($mol \cdot s^{-1}$) $\times 10^4$	CO_2 ($mol \cdot s^{-1}$) $\times 10^5$	CH_4 ($mol \cdot s^{-1}$) $\times 10^7$	CO_2 (%)	CH_4 (%)	CO_2 ($mol \cdot s^{-1}$) $\times 10^6$	CH_4 ($mol \cdot s^{-1}$) $\times 10^5$	CO_2 (%)	CH_4 (%)
1,000	3.41	3.41	9.27	1.95	99.79	0.21	248	34.1	42.14	57.86
800	2.73	2.73	8.87	2.05	99.77	0.23	184	27.3	40.32	59.68
700	2.39	2.39	8.73	2.10	99.76	0.24	151	23.9	38.84	61.16
600	2.05	2.05	8.41	2.11	99.75	0.25	121	20.4	37.10	62.90
500	1.71	1.71	8.05	2.18	99.73	0.27	90.1	17.0	34.59	65.41
400	1.36	1.36	7.50	2.33	99.69	0.31	61.5	13.6	31.12	68.88
300	1.02	1.02	6.58	2.51	99.62	0.38	36.6	10.2	26.42	73.58
200	0.68	0.68	5.10	2.82	99.45	0.55	17.4	6.79	20.41	79.59
100	0.34	0.34	2.95	3.32	98.89	1.11	4.88	3.38	12.63	87.37
50	0.17	0.17	1.62	3.65	97.80	2.20	1.17	1.67	6.54	93.46
25	0.09	0.09	0.87	3.81	95.80	4.20	0.17	0.82	2.02	97.98

We will run a simulation with the same configuration of the first experiment in table S1.

Feed			Permeate side				Retentate side			
Total flow rate ($mL \cdot min^{-1}$)	Molar flow rate		Molar flow rate		Composition		Molar flow rate		Composition	
	CO_2 ($mol \cdot s^{-1}$) $\times 10^4$	CH_4 ($mol \cdot s^{-1}$) $\times 10^4$	CO_2 ($mol \cdot s^{-1}$) $\times 10^5$	CH_4 ($mol \cdot s^{-1}$) $\times 10^7$	CO_2 (%)	CH_4 (%)	CO_2 ($mol \cdot s^{-1}$) $\times 10^6$	CH_4 ($mol \cdot s^{-1}$) $\times 10^5$	CO_2 (%)	CH_4 (%)
1,000	3.41	3.41	9.27	1.95	99.79	0.21	248	34.1	42.14	57.86
800	2.73	2.73	8.87	2.05	99.77	0.23	184	27.3	40.32	59.68
700	2.39	2.39	8.73	2.10	99.76	0.24	151	23.9	38.84	61.16
600	2.05	2.05	8.41	2.11	99.75	0.25	121	20.4	37.10	62.90

Inlet conditions.

CH4 molar mass: $M_{CH4} = 16.04 \times 10^{-3}\, kg/mol.$
CO2 molar mass: $M_{CO2} = 44.01 \times 10^{-3}\, kg/mol.$

CH4: $3.41 \times 10^{-4}\, mol/s = 5.47 \times 10^{-6}\, kg/s.$
CO2: $3.41 \times 10^{-4}\, mol/s = 1.50 \times 10^{-5}\, kg/s$.

In the OpenFOAM simulations, we use mass-based quantities (mass fraction, mass flow rate, mass permeance, $\cdots$), so we need to convert the molar quantities reported in the paper to mass quantities.

In the simulations, we enter the total mass flow rate in the inlet (feed) and the mass fraction of each component.

Total mass flow rate in the inlet (feed): $2.047 \times 10^{-5}\, kg/s$.

CH4 molar fraction: $X_{CH4} = 0.5$
CH4 mass fraction: $Y_{CH4} = 0.2671$

CO2 molar fraction: $X_{CO2} = 0.5$
CO2 mass fraction: $Y_{CO2} = 0.7329$

After the simulation, we convert back the results from mass to molar quantities, as the experimental data are reported in molar terms.

General relations between molar fraction (X_i) and mass fraction (Y_i). M_i is the molar mass of component i.

$$Y_i = \frac{X_i M_i}{\sum_j X_j M_j}$$

$$X_i = \frac{Y_i}{M_i \sum_j \left(\frac{Y_j}{M_j} \right)}$$

Permeances

The membrane permeance for each component is an input in our simulations.

We obtain the permeance values from the experimental data presented in the paper and run a simulation under the same conditions.

We then compare the **outlet flow rates and species concentrations.**

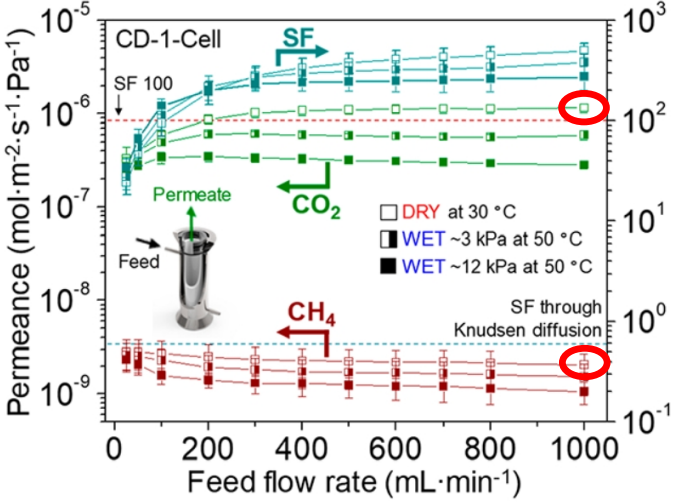
Our main goal is to compare the experimental and CFD results for CH4 concentration in the retentate outlet.

In summary: we will run the simulation using the same conditions as the experiment — including flow rate, concentration, pressure difference, geometry and permeances — and assess whether the retentate gas concentrations match the experimental results.

This comparison serves as a **validation** of our simulation.

If the **agreement** is good, we can confidently **extend the simulations to new conditions:** different geometries, pressures, gas compositions and more.

The permeances are obtained from the experimental data reported in the paper.



Permeances (mol):

$$P_{CH4} = 2.00 \times 10^{-9} \frac{mol}{m^2 \cdot s \cdot Pa}$$

$$P_{CO2} = 1.13 \times 10^{-6} \frac{mol}{m^2 \cdot s \cdot Pa}$$

Permeances (kg):

$$c_{CH4} = 3.21 \times 10^{-11} \frac{kg}{m^2 \cdot s \cdot Pa}$$

$$c_{CO2} = 4.97 \times 10^{-8} \frac{kg}{m^2 \cdot s \cdot Pa}$$

Note that the CO2 permeance is much higher than the CH4 permeance. This means that CO2 crosses the membrane more easily, enriching CH4 in the retentate.

The mass flux across the membrane is governed by the partial pressure difference:

$$S_{m,i} = \frac{c_i A}{V_c} \left(p_{i,R} - p_{i,P} \right)$$

$$p_{i,R} = X_i p_R \qquad\qquad p_{i,P} = X_i p_P$$

Where:

- $S_{m,i}$: mass transfer flux of component i across the membrane, per unit volume i $[kg/s]$
- c_i: membrane permeance of species i $[kg/(m^2 \cdot s \cdot Pa)]$
- A: membrane area $[m^2]$
- V_c: control volume $[m^3]$
- $p_{i,R}$: partial pressures on retentate side $[Pa]$
- $p_{i,P}$: partial pressures on permeate side $[Pa]$
- X_i: molar fraction of species i
- p_R: total pressure on the retentate side $[Pa]$
- p_P: total pressure on the permeate side $[Pa]$

Pressure

The gas separation process is **driven by a partial pressure difference** across the membrane.

In the experiments reported in the paper, the pressure on the retentate side of the membrane is $101\,kPa$, and on the permeate side it is $3\,kPa$. The pressure difference is therefore $98\,kPa$.

$$p_R = 101\,kPa$$

$$p_P = 3\,kPa$$

In the simulations, we will use *setFields* to impose a pressure of $101\,kPa$ in the retentate region and $3\,kPa$ in the permeate region.

Note: the experimental paper focuses on biogas purification. In natural gas purification, operating pressures are usually much higher - for example, $8\,bar$ on the retentate (feed) side and $1\,bar$ on the permeate side.

Membrane Size and Geometry

From this paragraph of the paper:

High-flux asymmetric α-alumina tubular supports (outer diameter 1.2 cm, thickness 0.2 cm, and length 9 cm; Finetech Co., Ltd., Republic of Korea) were used to fabricate zeolite films. $\cdots$. Before the zeolite film was synthesized, both ends of the α-alumina tubular support (ca. 2 cm), which were used for sealing in a cell or module, were glazed by impermeable materials (IN1001 Envision Glazes, Duncan Ceramics, USA).

we conclude that the membrane dimensions are:

- Outer diameter: $1.2\,cm$
- Total length: $9\,cm$
- Active zeolite length: $5\,cm$
- Impermeable support length: $2\,cm$ (on each end of the tube)

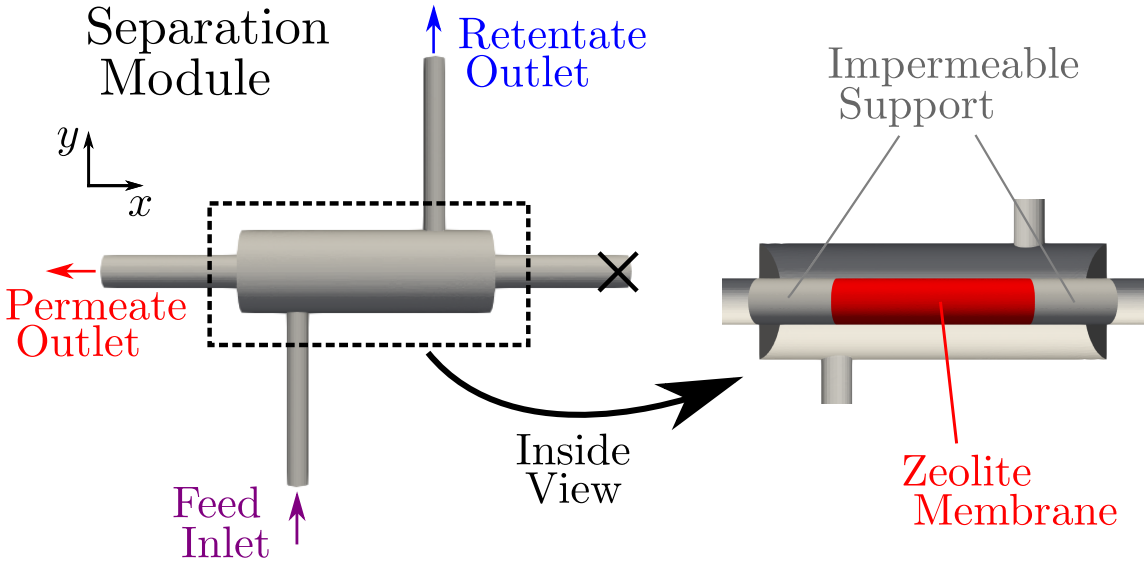

Separation
Module
Retentate
Outlet
y
x
Permeate
Outlet
Feed
Inlet
Inside
View
Impermeable
Support
Zeolite
Membrane

Table of Contents

Steps:

1. Geometry
2. Mesh
3. Case Setup
4. Running the Simulation
5. Post-processing

Geometry

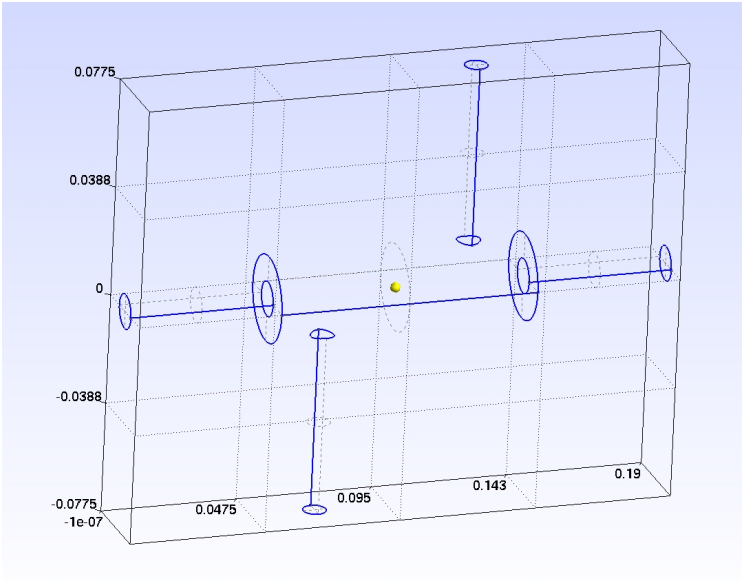
We need the geometry to generate the mesh for the simulation.

The geometry is created in Gmsh using `.geo` files.

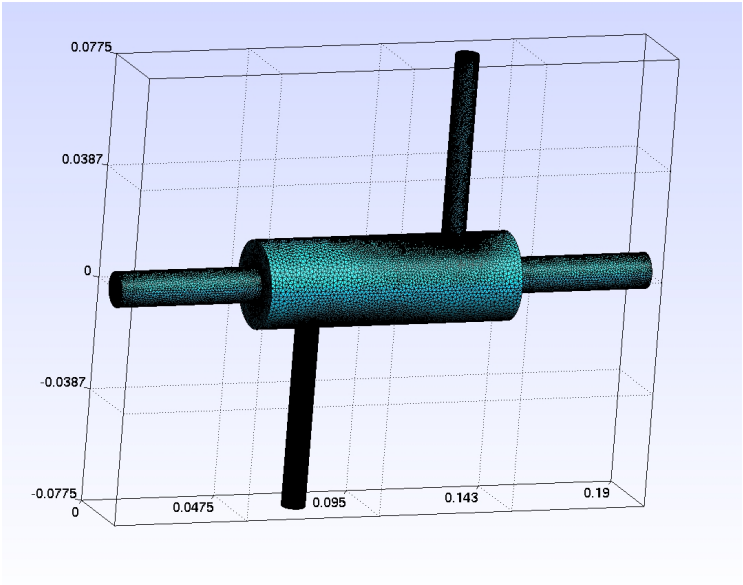
STL files are exported and placed in the `constant/geometry` folder.

We need to create separate STL files for the shell and for the internal walls (membrane and supports).

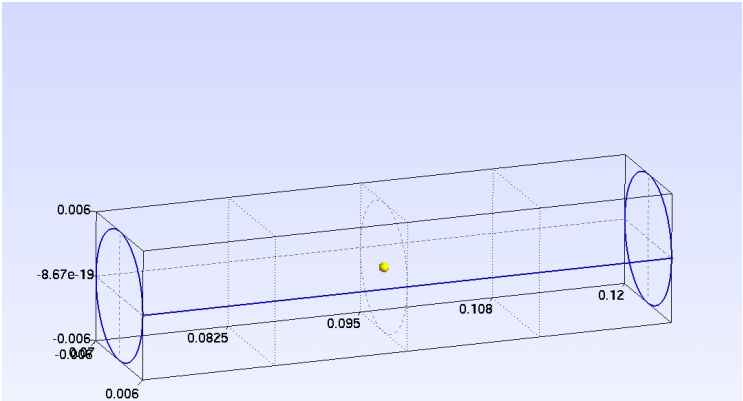
shell.geo



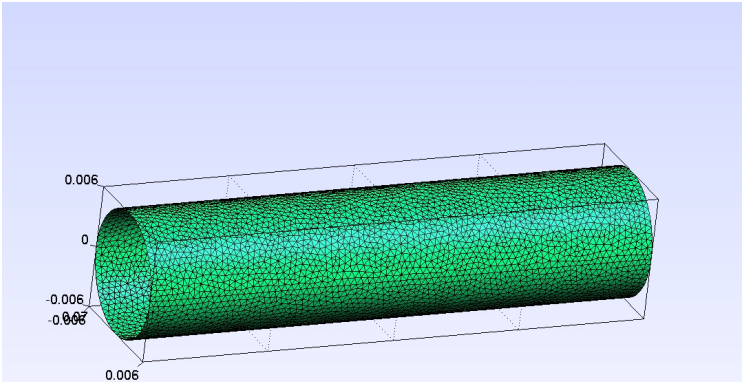
shell.stl



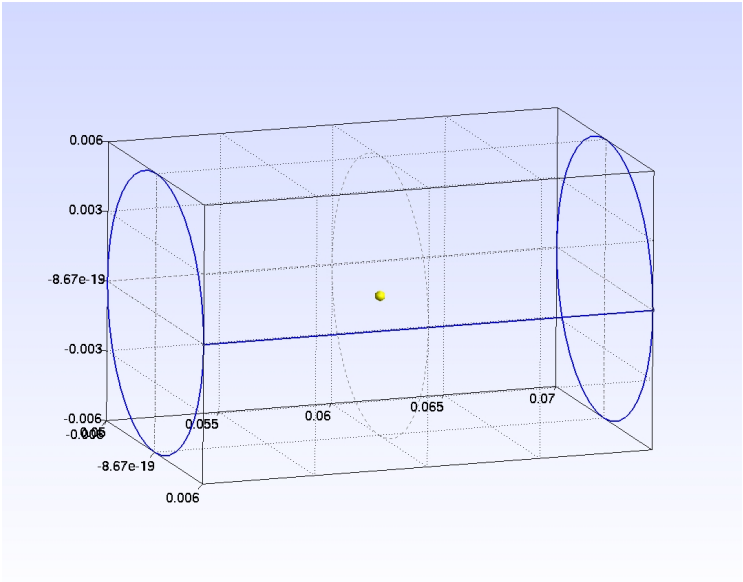
membraneZeolite.geo



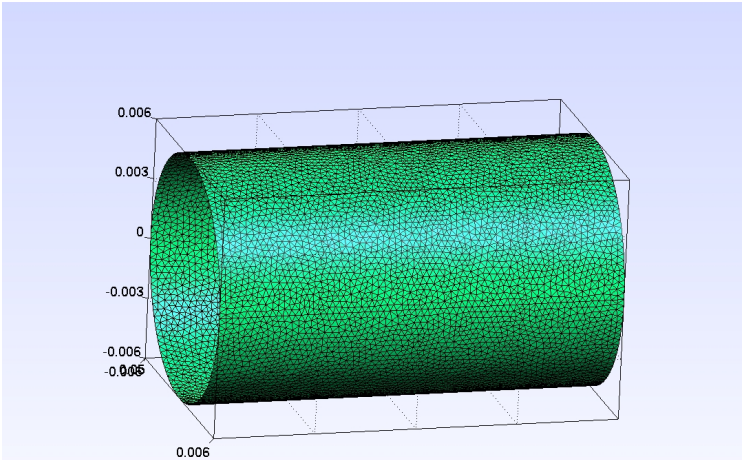
membraneZeolite.stl



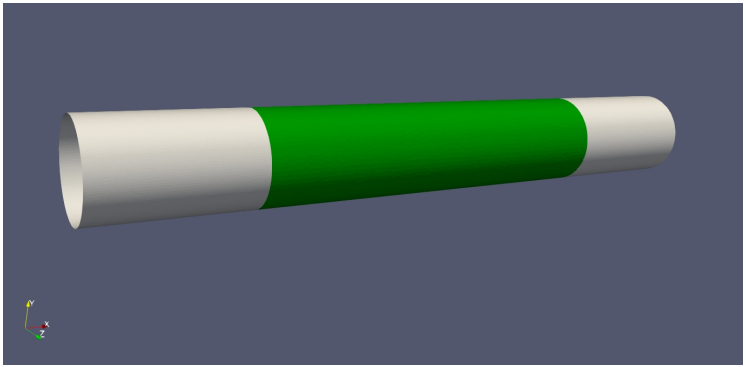
membraneSupport1.geo



membraneSupport1.stl



Zeolite membrane (green) and support



Mesh

To generate the mesh with snappyHexMesh we need

- the geometry (stl files in the constant/geometry folder),
- the background mesh
- and the snappyHexMesh configuration file.

The background mesh is generated using **blockMesh**.

The background mesh must fully enclose the geometry.

Use a regular Cartesian mesh (usually a single block).

Leave a margin around the geometry (e.g., 10–20% of the domain size).

Avoid highly stretched cells; prefer **cubic** ones.

Assign clear patch names for boundary conditions.

You can use a coarse mesh and make some refinement with **snappyHexMesh**.

system/blockMesh file:

```
convertToMeters 0.01;

vertices
(
    (-1.0 -9.0 -2.0) //0
    (20.0 -9.0 -2.0) //1
    (20.0 9.0 -2.0) //2
    (-1.0 9.0 -2.0) //3
    (-1.0 -9.0 2.0)  //4
    (20.0 -9.0 2.0)  //5
    (20.0 9.0 2.0)  //6
    (-1.0 9.0 2.0)  //7
);

blocks
(
    hex (0 1 2 3 4 5 6 7) (105 90 20) simpleGrading (1 1 1)
);
```

In this example, we generate a coarse mesh to reduce simulation time.

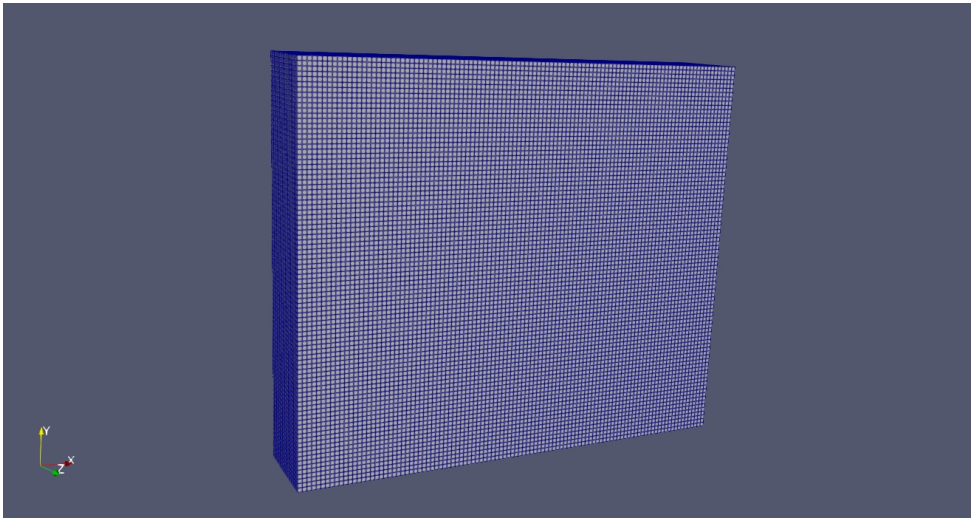
You can refine the background mesh by increasing the number of cells, for example:

```
convertToMeters 0.01;
blocks
(
    hex (0 1 2 3 4 5 6 7) (210 180 40) simpleGrading (1 1 1)
);
```

Note that the proportions are preserved.

Alternatively, you can use a coarse background mesh and apply local refinement with **snappyHexMesh**, using the `refinementRegions` option.

We generate the background mesh by running the *blockMesh* command.



We use the background mesh and the geometry (STL files) to generate the final mesh with **snappyHexMesh**.

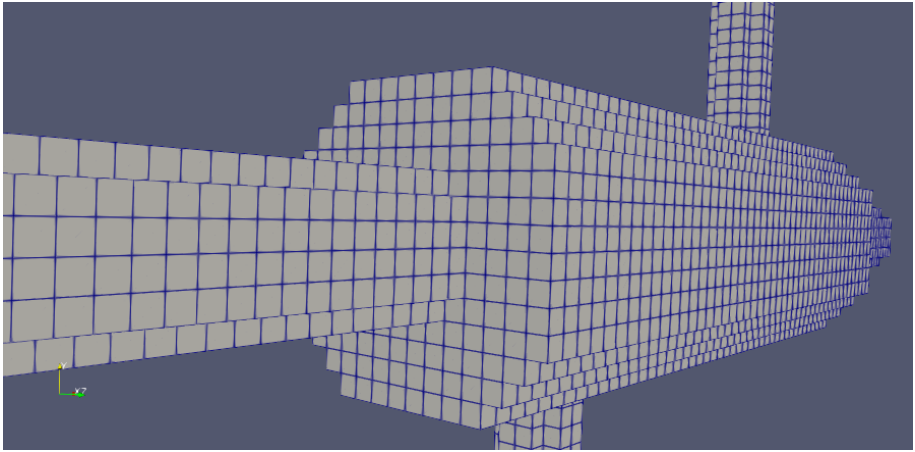
There are two main steps:

- *castellate*
- *snap*

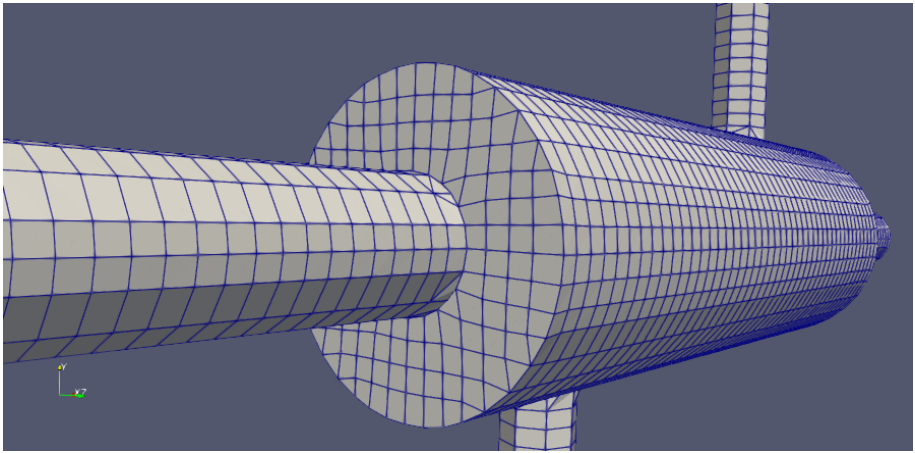
An optional third step, *addLayers*, can be included to add boundary layers, but we will not use it here.

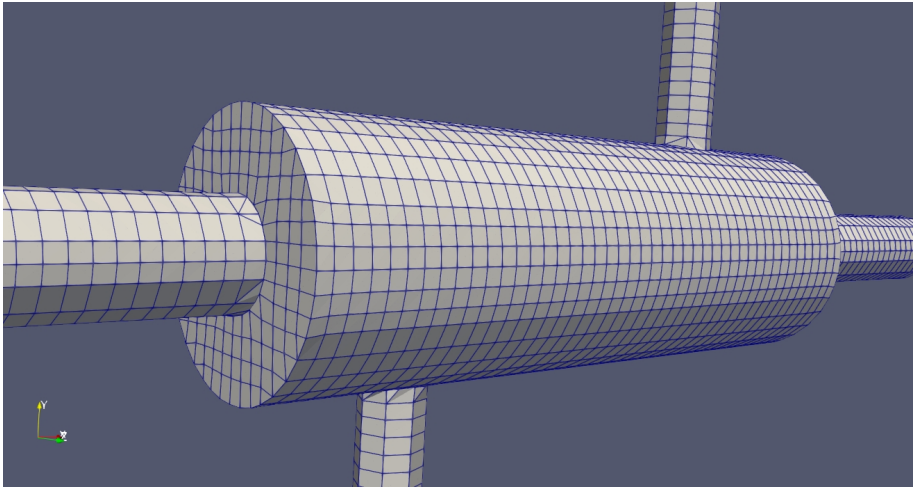
The *snappyHexMeshDict* file is located in the *system* folder. Details about its configuration are provided in Lecture 02.

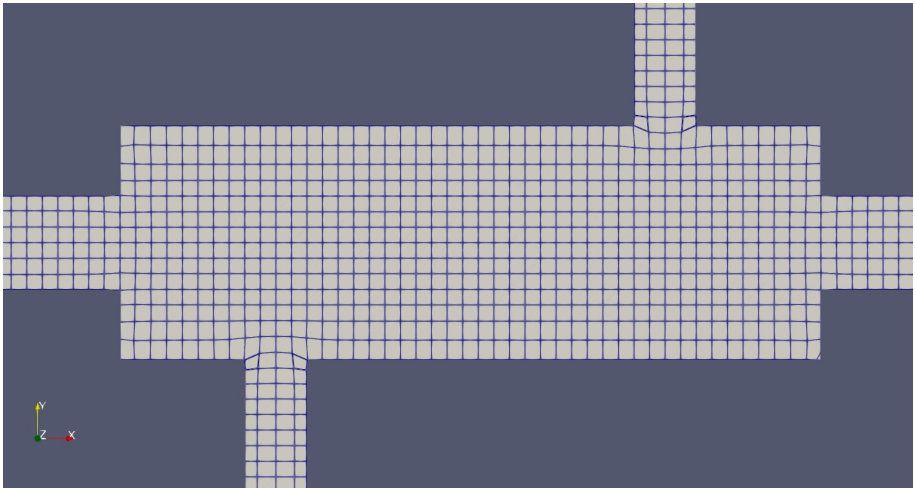
Run the command *snappyHexMesh*. This is the *castellate* step:

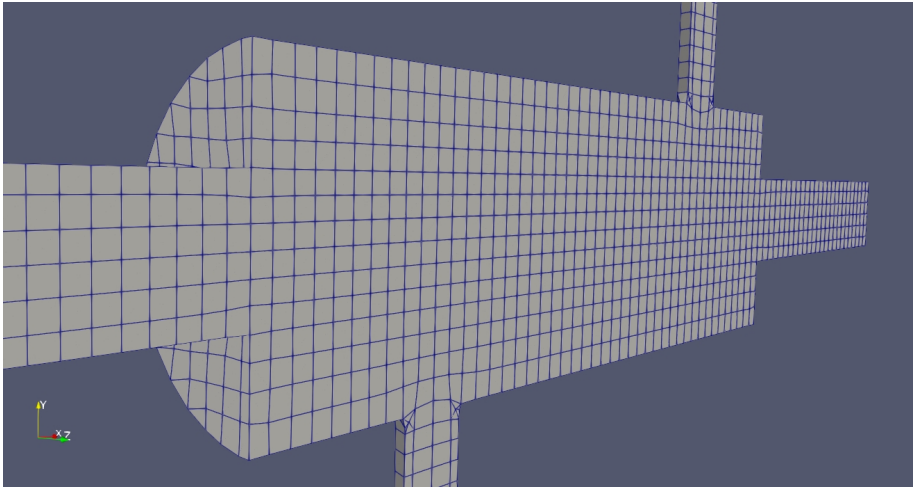


This is the *snap* step. This is the mesh that we are going to use in the simulation.




Y
X


Y
Z
X


Y
X
Z

Case Setup

We need to define the following parameters:

- Inlet concentrations of CH4 and CO2
- Membrane permeances
- Inlet flow rate
- Pressures at the feed and permeate sides
- Boundary conditions
- Gas properties

In the *0* folder, there is one file for each field variable.

The parameters can be defined directly in these files.

However, I have created **separate auxiliary files** to help with this setup.

The inlet concentrations are defined in the file *0/massFractionInlet*:

```
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "0";
    object      massFractionInlet;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

massFractionInlet
{
    CH4  0.2671;
    CO2  0.7329;
    N2   0.0;
}
```

These mass fractions correspond to a 50/50 molar ratio of CH4 and CO2.

The membrane permeances are defined in the file *0/permeances*:

```
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "0";
    object      permeances;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

{
    CH4  3.21e-11;
    CO2  4.97e-8;
    N2   0.0;
}
```

The inlet flow rate is defined in the file *0/massFlowRateInlet*:

```
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "0";
    object      massFlowRateInlet;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

massFlowRateInlet 2.0477e-5;
```

The pressures are defined in the file *0/pressures*:

```
1
2  FoamFile
3  {
4      format      ascii;
5      class       dictionary;
6      location    "0";
7      object      pressures;
8  }
9  // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
10
11 pressureRetentate     1.01e5;
12
13 pressurePermeate      3.e3;
14
15 pressureRetentateInlet uniform $pressureRetentate;
16
17 pressurePermeateOutlet uniform $pressurePermeate;
18
19 pressureRetentateOutlet uniform $pressureRetentate;
```

The boundary conditions for each variable are defined in its corresponding file.

A detailed discussion of boundary conditions can be found in Lectures 02 and 03.

The gas properties can be found in the *constant/physicalProperties* file:

```
thermoType
{
    type            hePsiThermo;
    mixture         multicomponentMixture;
    transport       sutherland;
    thermo          janaf;
    energy          sensibleEnthalpy;
    equationOfState perfectGas;
    specie          specie;
}

defaultSpecie CH4;

#include "physicalPropertiesValues"
```

A key boundary condition is the one applied at the membrane. From the *CH4* file:

```
membraneZeoliteTop
{
    type            semiPermeableBaffleMassFraction;
    samplePatch     membraneZeoliteBottom;
    c               $c;
    property        partialPressure;
    value           $internalField;
}
```

The membrane boundary condition (for U, p, T, *CH4*, and *CO2*) is responsible for enabling selective gas flow across the membrane.

Now we need to create the membrane (zeolite and supports) using the *createBaffles* command.

Refer to the *system/createBafflesDict* file. Lecture 02 presents the details of this process.

```
internalFacesOnly true;
baffles
{
    membraneZeolite
    {
        type       faceZone;
        zoneName   membraneZeolite;

        owner
        {
            name       membraneZeoliteTop;
            type       mappedWall;
            neighbourPatch membraneZeoliteBottom;
        }
```

We have the mesh generated, the case configured and the membrane created.

Before running the simulation, we need one final step: to impose the correct pressure on each side of the membrane.

This is done using the *setFields* utility.

Run the command *setFields*. The configuration can be found in the file *system/setFieldsDict*:

```
#include "../0/massFractionInlet"
#include "../0/pressures"

defaultFieldValues
(
    volScalarFieldValue p $pressureRetentate
);

regions
(
    zoneToCell
    {
        zone permeate;
        fieldValues
        (
            volScalarFieldValue p $pressurePermeate
        );
    }
);
```

Note 1: If you start the simulation without using *setFields*, it will lead to very small time steps due to large pressure differences in the flow. The inlet patch will be at $101\,\mathrm{kPa}$, while the rest of the domain will be at $3\,\mathrm{kPa}$. This creates an unstable simulation condition. Large pressure differences *across* the membrane are acceptable, but not on the *same side*.

Note 2: If the pressure difference across the membrane is very large, you can first run a simulation with a smaller pressure difference and use it as the initial condition. For example, if the feed pressure is $10\,\mathrm{bar}$ and the permeate pressure is $1\,\mathrm{bar}$, you can initially run a case with a $2\,\mathrm{bar}$ feed pressure and use the resulting velocity and concentration fields for the $10\,\mathrm{bar}$ case.

Copy all files from the *0* folder to the latest time folder.

Now let's take a look on the simulation configuration files.

The initial and final time can be defined in the *system/controlDict* file.

```
application     foamRun;

solver          multicomponentFluid;

startFrom       latestTime;

startTime       0;

stopAt          endTime;

endTime         20;

deltaT          1e-8;

writeControl    adjustableRunTime;

writeInterval   5;
```

```
18 purgeWrite      0;
19
20 writeFormat     ascii;
21
22 writePrecision 8;
23
24 writeCompression off;
25
26 timeFormat      general;
27
28 timePrecision  6;
29
30 runTimeModifiable true;
31
32 adjustTimeStep yes;
33
34 maxCo           1e-1;
35
36 maxDeltaT       1e-3;
37
38 libs            ( "libspecieTransfer.so" );
```

```
functions
{
    #includeFunc residuals

    #includeFunc patchFlowRateInlet
    #includeFunc patchFlowRateOutletPermeate1
    #includeFunc patchFlowRateOutletPermeate2
    #includeFunc patchFlowRateOutletRetentate

    #includeFunc patchAverageMassFractionInlet
    #includeFunc patchAverageMassFractionOutletPermeate1
    #includeFunc patchAverageMassFractionOutletPermeate2
    #includeFunc patchAverageMassFractionOutletRetentate

    #includeFunc patchAverageDensityInlet
    #includeFunc patchAverageDensityOutletPermeate1
    #includeFunc patchAverageDensityOutletPermeate2
    #includeFunc patchAverageDensityOutletRetentate
```

```
57      rhoFunc
58      {
59          type            writeObjects;
60          libs           ("libutilityFunctionObjects.so");
61          writeControl    adjustableRunTime;
62          writeInterval   $writeInterval;
63          objects        ("rho");
64      }
65
66      muFunc
67      {
68          type            writeObjects;
69          libs           ("libutilityFunctionObjects.so");
70          writeControl    adjustableRunTime;
71          writeInterval   $writeInterval;
72          objects        ("mu");
73      }
74
75  }
```

We are using the *multicomponentFluid* solver.

The simulation starts from the *latestTime* available (i.e., the time folder with the largest timestamp). This is important because the new meshes are saved as time steps, so OpenFOAM must start from the most recent one.

We run the simulation until $25\,\mathrm{s}$, which is sufficient for the inlet mass flow rate used here. However, smaller flow rates may require longer simulation times to reach convergence.

The initial *deltaT* is set to $10^{-8}\,\mathrm{s}$. It is important to start with a small time step like this. Later, *deltaT* is automatically adjusted according to the *maxCo* parameter.

Note all the functions included in the ***functions*** file.

These are calculations performed during the simulation and used later for post-processing.

When the simulation starts, a folder named *postProcessing* is automatically created.

We calculate the mass flow rate, the average mass fraction, and the average gas density at the inlet and outlet patches.

One of the most important results is the average mass fraction of CH4 at the retentate outlet.

These values are computed during runtime. You can see how each calculation is configured in the corresponding file inside the *system* directory.

Note that we also calculate the density (*rhoFunc*) and the viscosity (*muFunc*) within the domain.

We will return to these files in the post-processing section.

Numerical settings (integration methods and linear system solvers) for the simulation are defined in the *system/fvSchemes* and *system/fvSolution* files.

Detailed explanations of these configurations are provided in Lectures 01 and 02.

Running the Simulation

To run the simulation in **serial mode**, use the following command:

```
$ foamRun
```

To run the simulation in **parallel** using 4 processes, first run:

```
$ decomposePar
```

This will create four new folders in the case directory: *processor0*, *processor1*, *processor2*, and *processor3*.

Then execute:

```
$ mpirun -np 4 foamRun -parallel
```

The number of processes is defined in the *system/decomposeParDict* file. In this example, we are using 4 processes. Note that the number in the `mpirun` command must match this value.

```
FoamFile
{
    format      ascii;
    class       dictionary;
    location    "system";
    object      decomposeParDict;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

numberOfSubdomains 4;

method          scotch;
```

Other decomposition methods are available besides `scotch`. You are encouraged to explore them to find options better suited to your case.

Note 1: The number of processes in `mpirun` must match the value set in `numberOfSubdomains` inside `system/decomposeParDict`.

Note 2: Before running in parallel, always execute `decomposePar` to split the domain based on the chosen decomposition method.

Note 3: After the simulation, use `reconstructPar` to merge the results from all subdomains into a single dataset for post-processing.

Note 4: The optimum number of processes depends on the mesh size and your hardware. Using too many processes for a small mesh can increase communication overhead and slow down the simulation.

The simulation is now running. It may take some time to complete. (On my personal notebook, running in parallel with 4 processes, it took around 50 minutes.)

The first time step takes longer to execute due to the high number of iterations required to solve for CO2 and h. (In fact, the solver reaches the maximum number of allowed iterations without achieving the specified tolerance.)

After this initial step, the simulation proceeds more quickly.

First time-step.

```
Starting time loop

Courant Number mean: 0 max: 0
deltaT = 1.1999904e-06
Time = 4.19999e-06s

diagonal:  Solving for rho, Initial residual = 0, Final residual = 0, No Iterations 0
GAMG:  Solving for Ux, Initial residual = 1, Final residual = 6.7084716e-15, No Iterations 2
GAMG:  Solving for Uy, Initial residual = 1, Final residual = 1.0884838e-15, No Iterations 2
GAMG:  Solving for Uz, Initial residual = 1, Final residual = 1.6348469e-15, No Iterations 2
GAMG:  Solving for CO2, Initial residual = 0.002115057, Final residual = 0.00019855013, No Iterations 1000
GAMG:  Solving for N2, Initial residual = 0, Final residual = 0, No Iterations 0
GAMG:  Solving for h, Initial residual = 0.00098728529, Final residual = 9.2988754e-05, No Iterations 1000
DICPCG:  Solving for p, Initial residual = 4.4751735e-07, Final residual = 4.3143672e-09, No Iterations 1
DICPCG:  Solving for p, Initial residual = 1.0798893e-08, Final residual = 6.1067082e-11, No Iterations 1
diagonal:  Solving for rho, Initial residual = 0, Final residual = 0, No Iterations 0
time step continuity errors : sum local = 6.4549987e-11, global = -7.2468659e-13, cumulative = -7.2468659e-13
ExecutionTime = 0.517089 s  ClockTime = 1 s

Courant Number mean: 1.1315585e-06 max: 0.00056168936
deltaT = 1.4399712e-06
Time = 5.63996e-06s
```

Second time-step.

```
Courant Number mean: 1.1315585e-06 max: 0.00056168936
deltaT = 1.4399712e-06
Time = 5.63996e-06s

diagonal:  Solving for rho, Initial residual = 0, Final residual = 0, No Iterations 0
GAMG:  Solving for Ux, Initial residual = 0.29943236, Final residual = 7.2936169e-10, No Iterations 1
GAMG:  Solving for Uy, Initial residual = 0.20952747, Final residual = 6.8348354e-10, No Iterations 1
GAMG:  Solving for Uz, Initial residual = 0.31656604, Final residual = 1.63182e-16, No Iterations 2
GAMG:  Solving for CO2, Initial residual = 0.99999894, Final residual = 8.2015493e-11, No Iterations 2
GAMG:  Solving for N2, Initial residual = 0, Final residual = 0, No Iterations 0
GAMG:  Solving for h, Initial residual = 0.99999999, Final residual = 5.2942726e-13, No Iterations 2
DICPCG:  Solving for p, Initial residual = 6.1148308e-07, Final residual = 8.8101309e-09, No Iterations 1
DICPCG:  Solving for p, Initial residual = 1.9074934e-08, Final residual = 2.930502e-12, No Iterations 2
diagonal:  Solving for rho, Initial residual = 0, Final residual = 0, No Iterations 0
time step continuity errors : sum local = 3.0983896e-12, global = -1.6783006e-12, cumulative = -2.4029872e-12
ExecutionTime = 0.531622 s  ClockTime = 1 s
```

Post-Processing

Now we are going to analyze the simulation results.

It is useful to visualize contour plots of the main fields, such as velocity ($\mathbf{U}$), pressure (p), and molar fractions of CH4 and CO2. These provide a more qualitative understanding of the flow behavior.

In addition, we must generate line graphs of key properties over time (e.g., CH4 molar fraction at the retentate outlet) for a more quantitative analysis of the results.

The results are located in the `postProcessing` folder and in the latest time folder.

If you are running the simulation in parallel, execute:

```
$ reconstructPar -latestTime
```

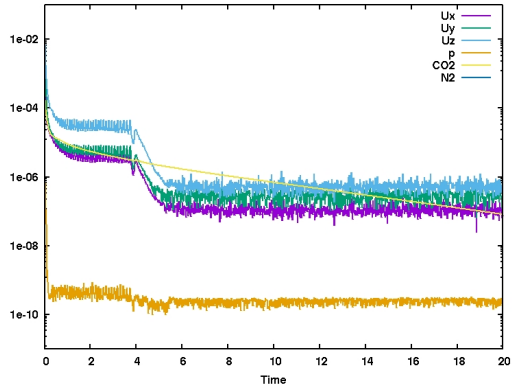
This command reconstructs the fields from the individual processor directories into a unified folder.

The residuals of the simulation can be visualized using the following command:

```
$ foamMonitor -l postProcessing/residuals/3e-08/residuals.dat
```

Note that the folder name 3e-08 may vary depending on the case.

This is an example of the residuals output:



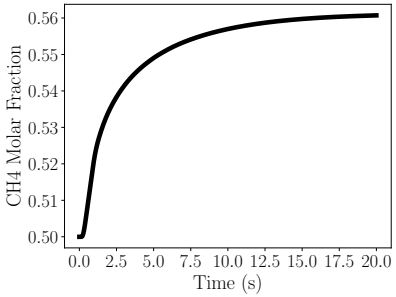
Note that the residuals start at high values and later oscillate around a small value, close to the solver tolerances. This indicates a good convergence behavior.

The script `scriptPostProcessResultInletOutlets.py` prints the final results (mass and molar concentrations) to the screen.

```
--------------------
--------------------
OutletRetentate

Gases: CH4, CO2, H2, N2, O2

Mass Fraction Y:  3.1751e-01, 6.8249e-01, 0.0000e+00, 0.0000e+00, 0.0000e+00
Mass Flow Rate: 1.7194e-05

Density: 1.1354e+00
Vol Flow Rate: 1.5144e-05 m3/s
Vol Flow Rate: 908.66 ml/min

Molar Fraction X:  5.6072e-01, 4.3928e-01, 0.0000e+00, 0.0000e+00, 0.0000e+00
Molar Flow Rate: 6.0701e-04
Molar Flow Rate of each Gas:  3.4036e-04, 2.6665e-04, 0.0000e+00, 0.0000e+00, 0.0000e+00
--------------------
--------------------
```

The CH4 molar fraction at the outlet is 56.07%, which is close to the experimental result of 57.86%.

The script `scriptPostProcessCH4MolarFractionRetentateOutletPlot.py` plots the CH4 molar fraction at the retentate outlet as a function of time.

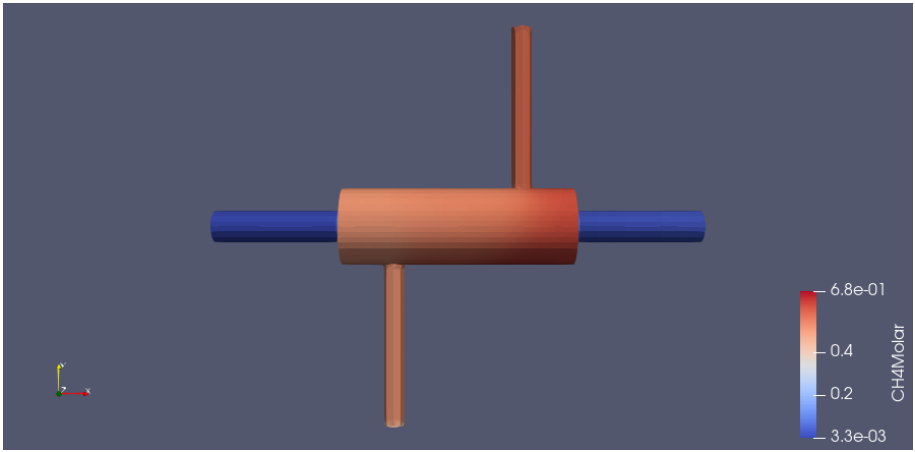


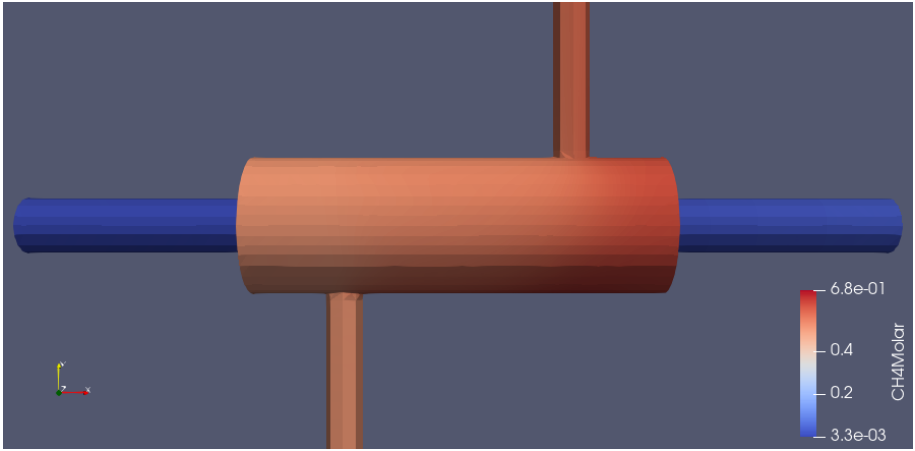
The simulation is nearly converged, but there is still a slight temporal variation in the CH4 molar fraction. Extending the simulation time would help ensure full convergence and improve the reliability of the final results.

Now we will visualize the flow using ParaView.

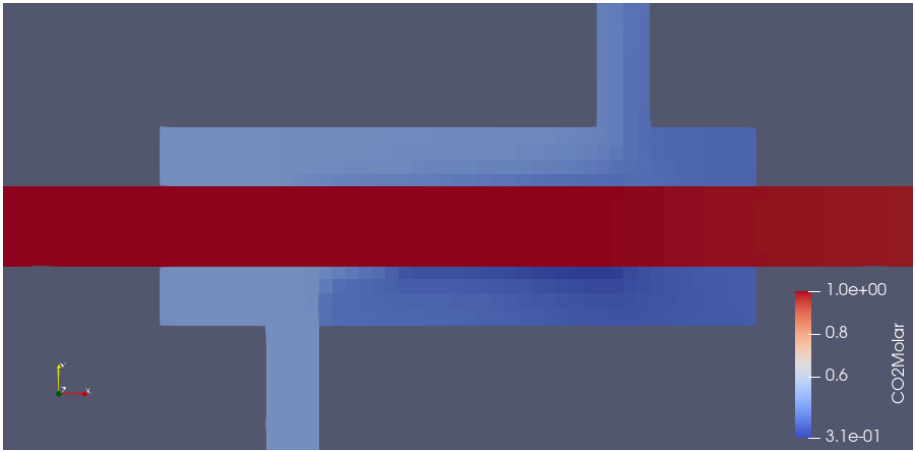
First, place the script `scriptPostProcessConvertToMolarFraction.py` inside the latest time folder (in our case, 20) and run it.

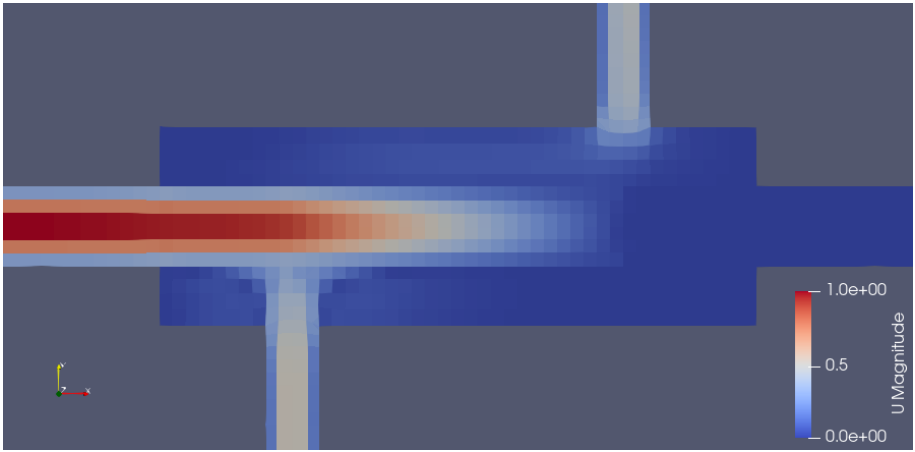
This will generate the files `CH4Molar` and `CO2Molar`, which we will use in the visualization.

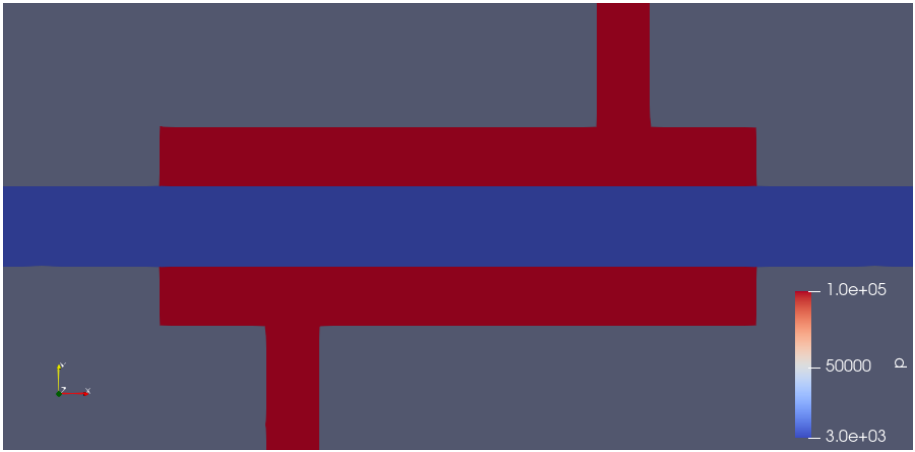

6.8e-01
0.4
0.2
3.3e-03
CH4Molar

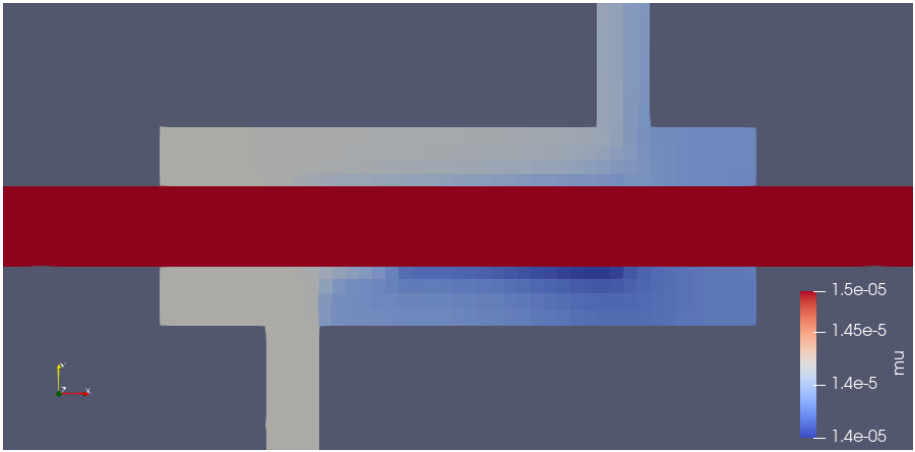

6.8e-01
0.4
0.2
3.3e-03
CH4Molar

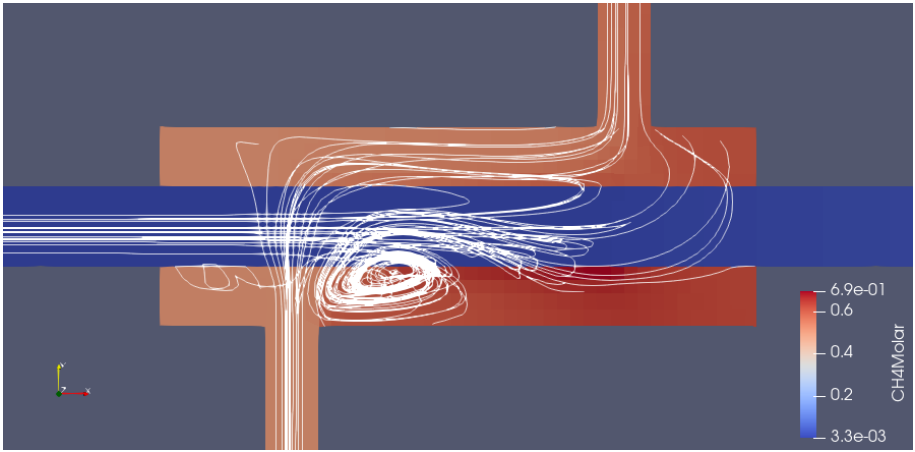

6.9e-01
0.6
0.4
0.2
3.3e-03
CH4Molar

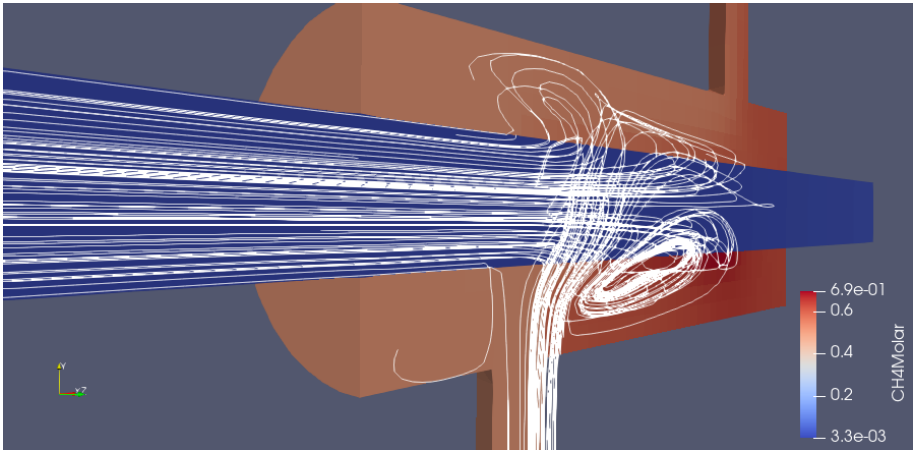

1.0e+00
0.8
0.6
3.1e-01
CO2Molar

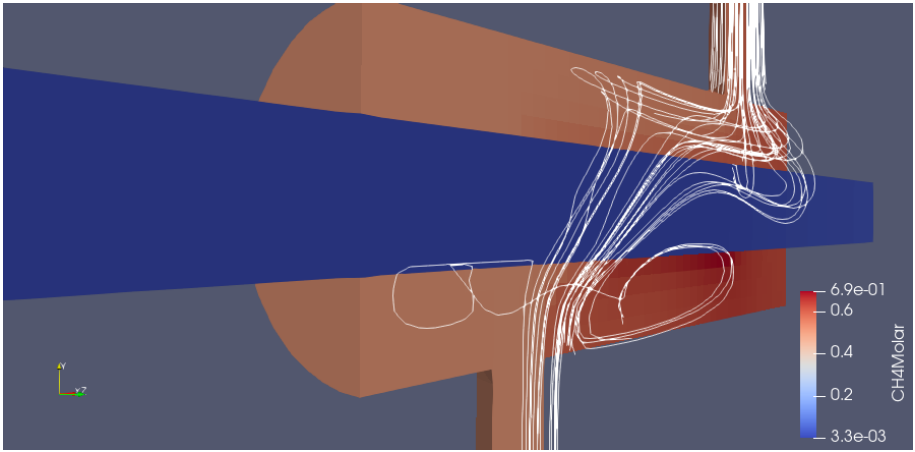

1.0e+00
0.5
0.0e+00
U Magnitude

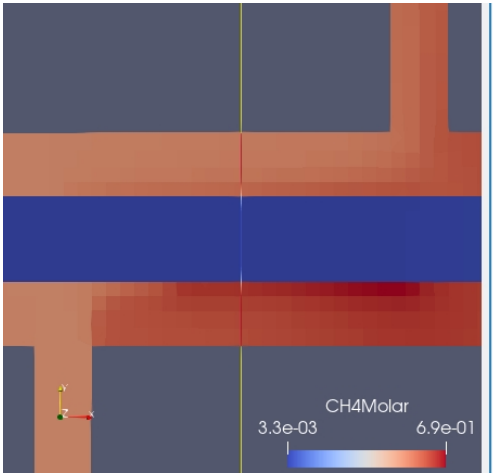

1.0e+05
50000
3.0e+03
p

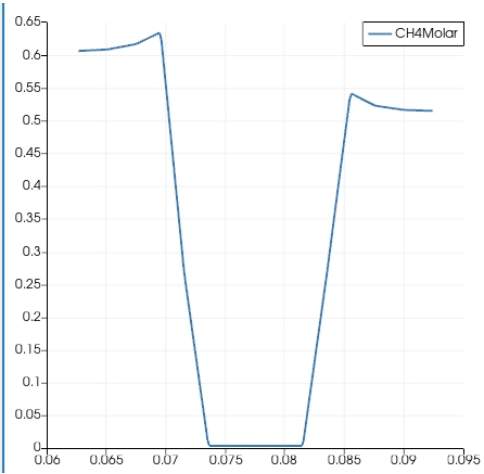

1.5e-05
1.45e-5
1.4e-5
1.4e-05
mu


6.9e-01
0.6
0.4
0.2
3.3e-03
CH4Molar


6.9e-01
0.6
0.4
0.2
3.3e-03
CH4Molar
Y
X
Z


6.9e-01
0.6
0.4
0.2
3.3e-03
CH4Molar
Y
X Z


CH4Molar
3.3e-03
6.9e-01


CH4Molar
0.65
0.6
0.55
0.5
0.45
0.4
0.35
0.3
0.25
0.2
0.15
0.1
0.05
0
0.06
0.065
0.07
0.075
0.08
0.085
0.09
0.095

Important note: if you run a simulation with a given inlet flow rate and want to test a different flow rate, you don't need to start from scratch.

You can use the final time-step from the first simulation as the initial condition for the second.

To do this, copy the last time folder from the first case into the new case directory and rename it as `0`. Make sure to also copy the corresponding mesh (i.e., the `polyMesh` folder).

Then, update the inlet flow rate directly in the `U` file — at the end of the file — by modifying the `massFlowRate` value.

Final Remarks

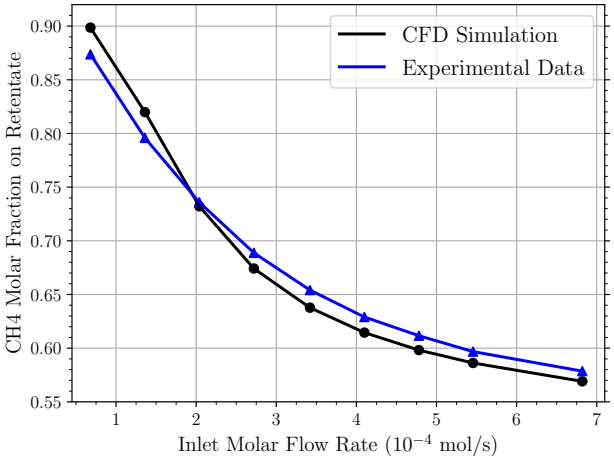
This concludes the complete simulation of the membrane-based gas separation module.

You are encouraged to explore additional results and create other visualizations.

The figures shown in the last slides are not in publication quality. You should refine them to improve their appearance.

You can enhance figure quality directly in ParaView or generate high-quality plots using Python scripts, for example.

This is a validation study performed using a mesh with approximately 500,000 cells.



To obtain reliable results, it is important to use a sufficiently refined mesh.

You can refine the mesh either in the `blockMeshDict` or in the `snappyHexMesh` file.

For this problem, a mesh with approximately 500,000 cells is typically adequate.

For publication purposes, it is important to include both a mesh convergence study and a validation against experimental or benchmark results.

Always check if the simulation has reached steady-state.

Monitoring quantities such as molar fraction or mass flow rate at the outlet helps determine if the results have stabilized.

Residuals alone are not enough — physical quantities should also stop changing significantly with time.

Well-designed visualizations make your results clearer and more convincing.

Use contour plots, streamlines and line graphs to highlight key features of the flow.

Adjust color scales, camera angles, and annotations in ParaView or use Python to produce high-quality, publication-ready figures.

Table of Contents

Here I present two recent papers that investigate the gas separation process in membrane-based modules using CFD simulations, along with a recent review article on the topic.

For more references, see the "Papers" section at the end of Lecture 00.

Link.

Chemical Engineering Science 302 (2025) 120864

Contents lists available at ScienceDirect

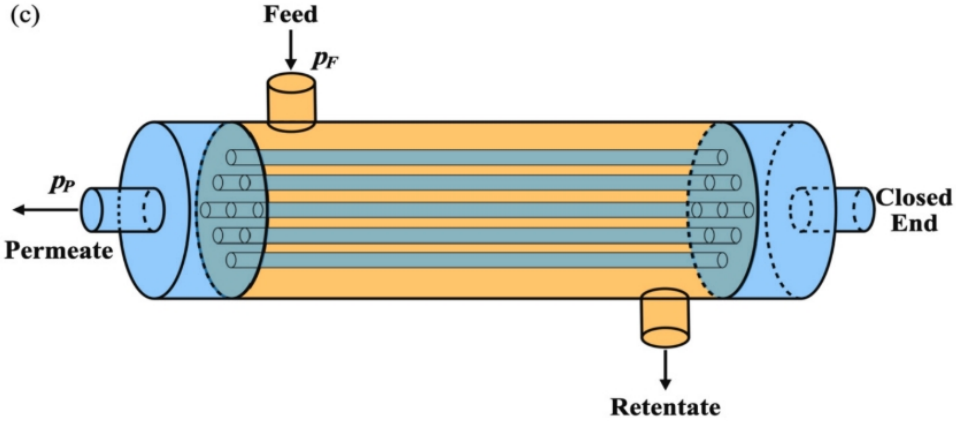
Chemical Engineering Science

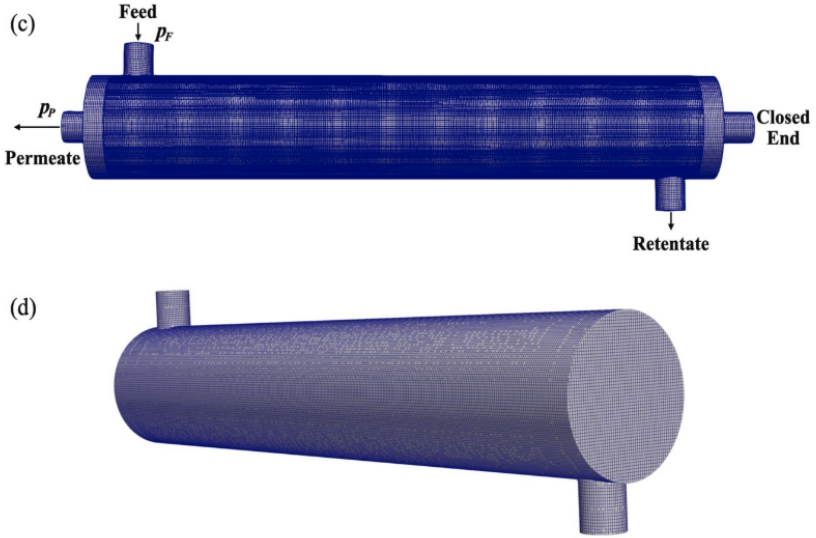
journal homepage: www.elsevier.com/locate/ces

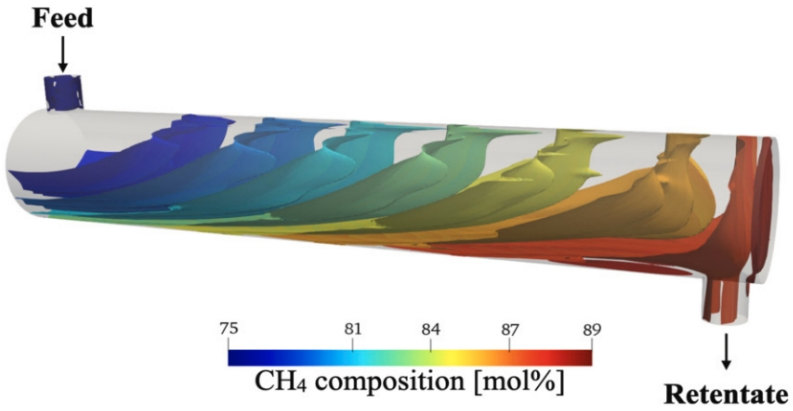
Three-dimensional CFD modeling for hollow fiber gas separation membrane modules based on a dual porous media model

Xiaobo Yao [a,b], Viatcheslav Freger [b], Xuezhong He [a], Ruofan Wang [a,b], Bo Kong [a,*]

[a] Department of Chemical Engineering, Guangdong Technion-Israel Institute of Technology, Shantou, 515063, Guangdong, China
[b] Faculty of Chemical Engineering, Technion-Israel Institute of Technology, Technion City, 3200003, Haifa, Israel


(c)
Feed
p_F
p_P
Permeate
Closed
End
Retentate


(c)
Feed
p_F
p_P
Permeate
Closed
End
Retentate
(d)


Feed
75
81
84
87
89
CH4 composition [mol%]
Retentate

Link.

Journal of Membrane Science 563 (2018) 199–209

Contents lists available at ScienceDirect

Journal of Membrane Science

journal homepage: www.elsevier.com/locate/memsci

Membrane modeling using CFD: Combined evaluation of mass transfer and geometrical influences in 1D and 3D

Bahram Haddadi*, Christian Jordan, Martin Miltner, Michael Harasek

Institute of Chemical, Environmental & Bioscience Engineering, TU Wien, Getreidemarkt 9/166, 1060 Vienna, Austria

ARTICLE INFO

Keywords:
Gas permeation
Multicomponent separation
Computational Fluid Dynamics
Process simulation
Module performance

ABSTRACT

In current literature two main approaches are used for the simulation of membrane contactors. One route considers membrane modules only in 1D for process simulation applications, the other route focuses on 3D simulation of modules using Computational Fluid Dynamics to provide very detailed information about membrane mass transfer or geometrical influences on the module performance.

A new CFD algorithm is introduced in the current work. It is capable of performing both 3D and 1D simulations using the same code – 1D to be used in fast process simulation applications whereas the 3D method can be applied for fully resolved CFD applications. Using experimental results from pure gas permeation of a hollow fiber module, it was demonstrated that 1D and 3D simulations compare with less than 2% deviation on a global scale. Based on the 3D simulations, it was found that the arrangement of the fibers can lead to high velocity zones close to the module walls. It was demonstrated that the 1D CFD method performs well even for almost pure gases like CH_4 at retentate side, by running simulations of a pilot scale biogas separation module in co- and counter-current configurations.

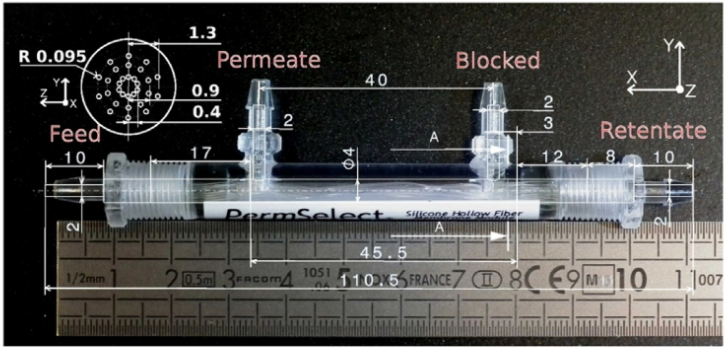
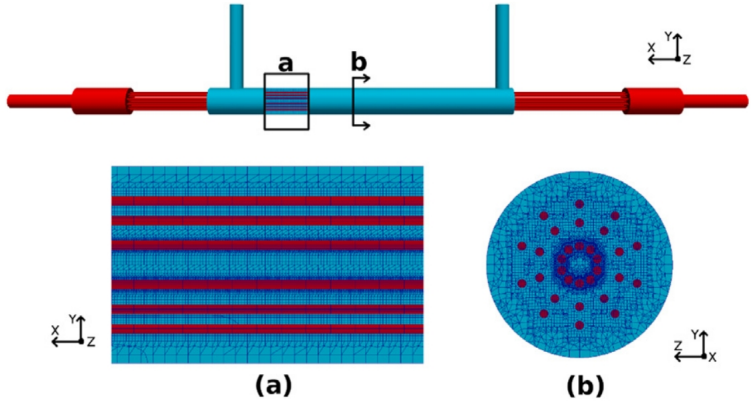


Fig. 3. Small membrane module and its dimensions (mm).


a
b
X
Y
Z
(a)
(b)

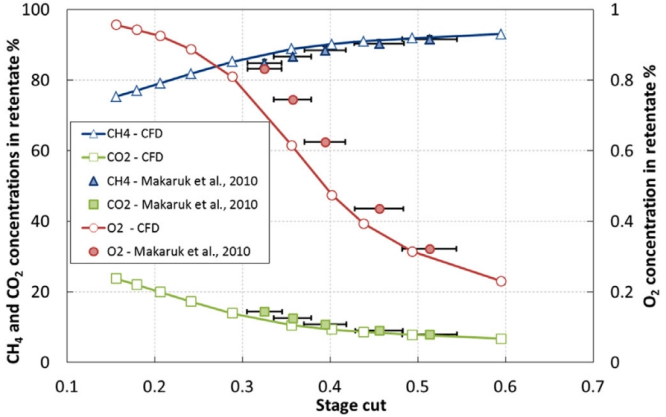


Fig. 12. Comparison of gas concentrations at different stage cuts in the retentate for co-current module: one-dimensional CFD code and experimental data [47] – oxygen is shown on the secondary y-axis.

Link.

Results in Engineering 26 (2025) 105531

Contents lists available at ScienceDirect

Results in Engineering

journal homepage: www.sciencedirect.com/journal/results-in-engineering

Review article

A comprehensive review of Computational Fluid Dynamics (CFD) modelling of membrane gas separation process

Muhammad Imran [a], Kok Keong Lau [a,*], Faizan Ahmad [b], Afiq Mohd Laziz [a]

[a] Centre of Carbon Capture, Utilisation and Storage (CCCUS), Department of Chemical Engineering, Universiti Teknologi PETRONAS, Bandar Seri Iskandar, 32610, Perak, Malaysia
[b] Centre for Sustainable Engineering, School of Computing, Engineering & Digital Technologies (SCEDT), Teesside University, Middlesbrough, TS17 5HT, United Kingdom

Table of Contents

Homework

Problem 1. Run the simulation with different inlet mass flow rates and compare the results with the experimental data.

Note: The simulation may take longer to converge at lower flow rates. You can monitor the CH_4 concentration at the retentate outlet to check if the simulation has reached a steady-state regime.

Table of Contents

Conclusion of Lecture 03

- Complete simulation of the membrane module
- Realistic setup based on experimental data
- Pressure and concentration boundary conditions
- Use of permeance to model membrane transport
- Validation of CH4 concentration in the retentate
- Extension to other pressures, gases and geometries

Conclusion of the Course (Topics)

- Membrane separation fundamentals
- Governing equations and transport models
- OpenFOAM workflow and structure
- Geometry: blockMesh and Gmsh
- Meshing: blockMesh, Gmsh and snappyHexMesh
- Simulation setup and execution
- Post-processing and data analysis
- Validation with experimental data
- Applications and further exploration

Conclusion of the Course (Text)

In this short course, we introduced the main concepts of membrane gas separation and explored how to simulate these systems using OpenFOAM.

We covered the physical principles behind membrane separation, including permeance, selectivity and mass transfer and studied the governing equations applied to CFD modeling.

From simple benchmark cases to the complete simulation of a 3D zeolite membrane module, we developed skills in geometry creation, mesh generation, solver setup, boundary conditions and post-processing techniques.

Finally, we validated our simulations against experimental data, reinforcing the importance of modeling assumptions, numerical settings and physical understanding.

You are now ready to explore new configurations, optimize designs and contribute to research and development in membrane-based gas separation systems using open-source tools.